

Home Lab Project Week 6, Day 2

Task:

Capture the Nmap scan traffic with Wireshark while repeating the scan. Save the PCAP file and identify the traffic generated by the scan.

Capturing & Identifying Nmap Scan Traffic

Engagement: Home Lab Internal Network Assessment **Lab log:** Week 6, Day 2

Target: Ubuntu 192.168.78.5 · **Tester:** Kali Linux 192.168.78.3 **Prepared by:** [Mostafa], DAE Cybersecurity Program **Date:** [Fill in]

1. Objective

My objective was to capture the Nmap scan traffic in Wireshark while repeating the SYN scan from Week 6 Day 1, save the PCAP file, and identify the traffic the scan generated.

2. Capture Setup

I started the capture *before* running the scan so no packets were missed:

1. In Wireshark on Kali, I selected the **eth1** interface (the host-only network — **eth0** is NAT and would not see this traffic).
 2. I set the capture filter to **host 192.168.78.5** so I only recorded traffic to and from the target.
 3. I started the capture.
-

3. Scan I Repeated

With the capture running, I re-ran the SYN scan from Kali:

```
sudo nmap -sS 192.168.78.5
```

After the scan finished, I stopped the capture.

4. Saving the PCAP

I saved the capture with **File** → **Save As**, as `week6-day2-nmap-scan.pcapng`. (To keep only the scan traffic, I could also apply a display filter first and use **File** → **Export Specified Packets** → **Displayed**.)

Evidence: `week6-day2-nmap-scan.pcapng`

5. Identifying the Scan Traffic

A SYN scan has a distinctive signature. I used the following display filters to isolate each part of it.

5.1 The SYN flood (the scan probes)

```
tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.src == 192.168.78.3
```

This shows Kali sending a large number of **SYN packets** to many different destination ports on the target in rapid succession. This fan-out — many ports, one SYN each, in a very short time from a single source — is the core signature of a port scan.

5.2 Closed ports (RST responses)

```
tcp.flags.reset == 1 && ip.src == 192.168.78.5
```

For each closed port, the target replies with a **RST** packet. Since 998 ports were closed, this is the bulk of the target's responses.

5.3 Open ports (SYN-ACK responses)

```
tcp.flags.syn == 1 && tcp.flags.ack == 1 && ip.src == 192.168.78.5
```

Only the open ports reply with **SYN-ACK**. I expect to see exactly two of these — from **port 21 (FTP)** and **port 23 (telnet)** — matching the open ports found in Day 1.

5.4 The "half-open" behavior

The reason it's called a half-open scan is visible here. For an open port, the exchange is:

```
Kali → Target : SYN      (probe)
Target → Kali : SYN, ACK  (port is open)
Kali → Target : RST      (Kali aborts instead of completing the handshake)
```

Kali sends **RST** instead of the final ACK, so the three-way handshake is never completed and no full connection is established. This is what makes the scan fast and stealthier than a full-connect scan — and it's the clearest way to identify a SYN scan in the capture.

5.5 Quantifying the scan

I used **Statistics** → **Conversations** (TCP tab) to see the volume: a single source (**192.168.78.3**) opening brief exchanges with ~1,000 different ports on one destination is unmistakably a scan, not normal traffic. **Statistics** → **Protocol Hierarchy** confirmed the capture is almost entirely TCP.

6. Observations

- The capture clearly shows the scan pattern: a burst of SYN packets from Kali to ~1,000 ports on the target.

- The target returned **RST** for the 998 closed ports and **SYN-ACK** for the 2 open ports (21 and 23).
 - For the open ports, Kali responded with **RST** rather than completing the handshake — confirming the half-open (SYN) scan behavior.
 - From a defender's viewpoint, this traffic is a textbook example of what a port scan looks like and is exactly what an intrusion-detection system would flag.
-

7. Evidence Log

- [] [week6-day2-nmap-scan.pcapng](#) — full capture of the scan
 - [] Screenshot: SYN flood (filter 5.1)
 - [] Screenshot: RST responses from closed ports (filter 5.2)
 - [] Screenshot: SYN-ACK from open ports 21 and 23 (filter 5.3)
 - [] Screenshot: **Statistics** → **Conversations** showing the fan-out across ~1,000 ports
-

This document records authorized security testing I performed in a controlled lab environment.